

Linux XFRM asynchronous SA management

A code walkthrough and study guide for telecom systems engineers

Review date: 22 September 2026. Repository: rizwan3659/xfrm-async-sa. Baseline reviewed: 4a31962.
Read this guide alongside the review branch.

This project demonstrates how a C userspace program installs IPsec security associations through Linux NETLINK_XFRM. Its central experiment compares waiting for every acknowledgement with keeping several requests outstanding. It is small enough to understand completely, including the parts a production implementation would still need.

The strongest technical story is about request ownership, bounded outstanding work, error isolation, and recovery after uncertain outcomes. The benchmark is supporting evidence, not the whole engineering story. This project does not implement a kernel module, a complete P-CSCF, key negotiation, or an encrypted packet test.

Reading map

- 1 A map of the project
- 2 IPsec concepts you need first
- 3 Where this fits in IMS
- 4 The Netlink wire contract
- 5 Building an SA message
- 6 Reading the scheduling loop
- 7 Failure semantics and uncertain outcomes
- 8 Backpressure and transport behavior
- 9 Review findings and repairs
- 10 What the tests establish
- 11 Running the laboratory
- 12 Measuring performance honestly
- 13 Extending this toward a service
- 14 Interview explanation and practice
- 15 A seven session learning plan
- 16 References and further reading

2 IPsec concepts you need first

An SA is a unidirectional security context. It includes identifiers, algorithms, keying material, and replay/lifetime state. One SA does not create a bidirectional secure connection. In this lab the supported case is IPv4 ESP in transport mode.

A policy says what to do with matching traffic. A state supplies the security context used for a transformation. Installing a state without a matching policy does not demonstrate that application packets are protected. This distinction is fundamental to debugging a system where `ip xfrm state` looks correct but traffic is still cleartext or rejected. See RFC 4301 [R1].

The SPI identifies an SA in its lookup context; it is not a secret or a globally unique identifier. For this restricted mock, the identity is destination address plus SPI because every entry has the same address family and ESP protocol. A general implementation needs a fuller key and must account for marks and other lookup context.

The `reqid` is a separate association identifier useful for relating state and policy templates. It is neither the SPI nor the Netlink sequence number. Netlink sequences correlate control requests; ESP sequence numbers support packet replay protection. Confusing these three identifiers leads to incorrect designs.

Control plane: application -> Netlink -> install SA and policy
Data plane: packet -> policy/state lookup -> ESP transform

Netlink sequence: which request does this ACK complete?
SPI: which security association handles this packet?
ESP sequence: is this packet fresh or a replay?

Transport mode protects the upper-layer payload while retaining the original IP header. Tunnel mode carries an inner IP packet within a new outer packet. This repository implements only the transport-mode state builder; it does not offer a mode switch.

Checkpoint: Does a successful NEWSA ACK prove a remote peer can decrypt traffic? No. It proves that the local kernel accepted that state operation. Peer state, policies, routing, keys, selectors, and actual packets still require validation.

3 Where this fits in IMS

The relevant telecom boundary is UE to P-CSCF access security. IMS security setup involves authentication-derived key material and negotiated security parameters. The conventional client/server protected-port arrangement uses two pairs of unidirectional SAs. Policies must bind the appropriate traffic selectors. Consult the applicable release and access scenario in TS 33.203 [R2]; this project is not a conformance implementation.

The `sa_spec` structure represents one state, not one complete UE registration. The benchmark generates N unrelated state requests. Therefore SA/s cannot be renamed registrations/s. Registration completion also depends on signaling, authentication, state/policy installation, and any rollback needed after partial failure.

Conceptual IMS control flow, not implemented here

Registration and authentication

- > negotiate security parameters
- > derive/obtain keys
- > install required states and policies
- > continue protected signaling

The hard-coded AES-GCM algorithm is a Linux XFRM example. It does not prove that an IMS peer has negotiated that algorithm or that a particular deployment permits it. Likewise the generated addresses, SPIs, and deterministic keys are laboratory data. They must not be used for operational traffic.

For an experienced telecom engineer, connect this code to a real design decision: a control process should track asynchronous state operations without losing the registration context that requested them. A production context might contain the UE identity, registration generation, required SAs/policies, completion count, and rollback state. None of those application-level fields currently exists in `sa_spec`.

Exercise: Draw a registration with four state operations. Let three succeed and one fail. Specify when signaling may proceed, which successful objects must be removed, and how a late ACK changes the rollback plan. Do not assume the whole set is an atomic kernel transaction.

4 The Netlink wire contract

NETLINK_XFRM is a Classic Netlink protocol. This code opens `socket(AF_NETLINK, SOCK_RAW | SOCK_CLOEXEC, NETLINK_XFRM)`, binds a local endpoint, and connects to the kernel endpoint. No Generic Netlink family-name lookup or `genlmsg_hdr` is used.

Each request starts with `nlmsg_hdr`, followed by an XFRM payload and any attributes. `nlmsg_len` includes the header. `nlmsg_type` selects NEWSA or DELSA. `NLM_F_REQUEST` | `NLM_F_ACK` requests an explicit result. The kernel returns the request sequence in its response [R3].

NEWSA

```
[ nlmsg_hdr ]  
[ xfrm_usersa_info ]  
[ nlattr: XFRMA_ALG_AEAD ]  
[ xfrm_algo_aead + key bytes ]
```

ACK

```
[ nlmsg_hdr: NLMSG_ERROR, matching sequence ]  
[ nlmsgerr: error + original request header ]  
[ optional echoed data / extended ACK attributes ]
```

Despite its name, `NLMSG_ERROR` also reports success: `error == 0`. Negative values carry `errno`-style failure information. One received datagram can contain multiple messages, so `drain_acks` walks the payload rather than assuming one receive equals one ACK.

The parser copies fixed headers into aligned local structures before reading them. It rejects short buffers, inconsistent message lengths, and invalid positive/out-of-range error values. Extended ACK diagnostics are not decoded. A datagram truncated by the receive buffer is reported as `EMSGSIZE` by the transport rather than accepted as a complete response.

Checkpoint: Why is `send` success insufficient? The `socket` operation and the XFRM operation have different results. The request may have been delivered but rejected by XFRM; the ACK carries that operation result.

5 Building an SA message

In `xfrm_build_newsa`, start at `NLMSG_LENGTH(sizeof(struct xfrm_usersa_info))`. Check capacity before writing, clear the base structure, then fill the header and XFRM fields. The caller supplies an eight-byte-aligned buffer because the payload includes fields with stronger alignment requirements than the Netlink header alone.

The input SPI is host order and is converted with `htonl`. Addresses already come from `inet_pton` or an explicit network-order conversion. Fields such as `reqid`, `lengths`, `flags`, and algorithm bit counts follow the local Linux UAPI representation. Do not apply `htonl` indiscriminately to every integer.

The builder sets IPv4, ESP, transport mode, a replay window of 32, and unlimited byte/packet limits. Time-lifetime fields remain zero. That is sufficient for this install/delete experiment; it does not provide a rekey policy or complete SA lifecycle manager.

The AEAD attribute contains the algorithm name `rfc4106(gcm(aes))`, an authentication-tag size of 128 bits, and 20 bytes of key material. Those 20 bytes mean a 16-byte AES key plus four bytes of salt, not a 160-bit AES key. This layout follows the RFC 4106 construction and the Linux algorithm API [R4, R5].

`put_attr` starts the attribute at the aligned end of the base message. Its attribute length counts the attribute header and payload; the outer message length includes alignment padding. Capacity checks include the aligned attribute size.

`xfrm_build_delsa` is shorter: it supplies destination, SPI, family, and protocol in `xfrm_usersa_id`. There is no key in a delete request. This also explains why careless cleanup can target a state installed by another process using the same identity.

Exercise: For SPI `0xABCD`, find the network-order conversion in the code, then locate the matching assertion in `test_newsa_encoding`. Explain why the algorithm key length is 160 while the AES key is 128 bits.

6 Reading the scheduling loop

All three public operations call `run`. Synchronous installation is the same machinery with window one; asynchronous installation uses the requested window; deletion changes the builder. This avoids maintaining separate correlation and timeout logic for each operation.

At entry, the function validates inputs, allocates per-request deadlines, and reserves a fresh sequence range on the transport. Results begin as `SA_PENDING`. The first request uses `base`; request `i` uses `base + i`. The transport retains its sequence counter across install and delete batches.

```
while unfinished work exists:
    send until the window is full or the socket pushes back
    expire sent requests whose monotonic deadline has passed
    find the nearest outstanding deadline
    receive and process every valid matching ACK in one datagram
    refill the window on the next iteration
```

`next` is the count of requests sent successfully. `inflight` counts sent requests with no terminal result. `done` counts completed or timed-out requests. `first` advances past completed entries to reduce repeated scanning. A valid ACK must reference a request below `next`, in this batch, whose result is still pending.

The invariants are `done + inflight == next`, `next <= n`, and `inflight <= window`. An ACK for an unsent request cannot decrement `inflight`. A duplicate cannot complete an already completed slot. A stale response from a previous batch cannot match the new sequence range.

The implementation is single-caller per transport. The counter is not atomic, and the socket has no response dispatcher for concurrent callers. Separate threads must serialize calls or use separate transports. Sequence exhaustion produces `E_OVERFLOW`; reopening the socket creates a fresh response domain instead of wrapping into old identifiers.

Exercise: Trace `N=3`, `W=2`. Send sequences 1 and 2; receive ACK 2; send 3; receive ACK 2 again; receive ACK 1 and ACK 3. Record the three counters at each step.

7 Failure semantics and uncertain outcomes

There are three different result categories. A zero result means the manager observed a success ACK. A negative kernel errno means it observed a rejection. -ETIMEDOUT means it did not observe a valid matching ACK before the deadline. It does not prove that the kernel made no change.

On a fatal transport/local error, the batch returns -1 with errno. Completed entries remain available. Pending entries may be unsent or sent with unknown outcome; the public interface does not expose a separate sent bitmap. Callers must not interpret every pending slot as a failed kernel operation.

```
Request sent -> kernel installs state -> ACK lost
                    |
                    caller times out
```

Blind retry -> EEXIST

Blind delete -> may affect an object whose ownership is uncertain

The review fixes two correlation hazards. Previously every batch reused the same sequence base, allowing a late install ACK to appear to complete a delete. Also, any datagram producing zero accepted ACKs was treated as a timeout. The updated code maintains fresh sequences and bases expiry on monotonic time, so unrelated traffic neither causes immediate failure nor extends a deadline indefinitely.

Each deadline starts after a successful send returns. This is not a deadline for the entire syscall or the whole UE registration. Netlink request processing can run inside send, and CPU scheduling can delay userspace progress. A production service needs a separately defined end-to-end budget.

For recovery, add an explicit reconciliation phase: query state, compare identity and ownership metadata, then decide whether to accept, retry, replace, or remove it. The repository does not implement GETSA/dump reconciliation. That limitation is deliberate and visible.

Checkpoint: Should -EEXIST be treated as success? Only after verifying that the existing object is the intended state with the expected parameters and ownership. This benchmark treats it as failure.

8 Backpressure and transport behavior

A window is an upper bound on outstanding work, not a guarantee that the socket can buffer that many messages. The original blocking send loop could fill the request queue while the worker filled the reply queue. Both sides could then wait for the other to drain, with the manager not yet receiving.

The updated sender uses `MSG_DONTWAIT`. If it gets `EAGAIN` while requests are outstanding, the manager stops filling and drains replies before trying again. If no request is outstanding, `EAGAIN` is reported as a transport failure rather than spinning without a useful event source. A service could extend this with `POLLOUT` and a send deadline.

`MSG_NOSIGNAL` prevents a disconnected mock peer from terminating the process with `SIGPIPE`. `recv_timeout` uses `poll`, a monotonic deadline, and `recvmsg`. Signal interruptions do not restart a full fresh timeout. Peer closure and truncated datagrams become errors. Socket buffer enlargement is only a performance hint; correctness no longer depends on it succeeding.

The mock has one worker thread and a socket pair using `SOCK_SEQPACKET`, which preserves datagram boundaries. Only the worker accesses the SA table. The installed count is read by another thread, so relaxed atomic operations protect that scalar. Relaxed ordering is sufficient for the counter as a standalone measurement; it does not publish access to the table.

Shutdown closes the client side, joins the worker, then frees the table and transport. Joining matters: freeing the table while the worker still runs would be a use-after-free. ThreadSanitizer checks exercised races, but it is not a proof of correct lifecycle behavior for every possible schedule.

Exercise: Remove `MSG_DONTWAIT` in a disposable branch, reduce socket buffers, and use a large window with an external test timeout. Explain the two queues and the resulting wait cycle before running anything.

9 Review findings and repairs

The reviewed baseline was 4a31962. The following are source-level findings addressed by the review branch; regression coverage is in the tests referenced below.

- **High: cleanup ownership.** Real mode deleted every requested identity, including EEXIST failures. It now deletes only ACK-confirmed successful installs, reports incomplete cleanup, and skips the second real batch after failure. `test_real.sh` pre-installs a colliding SA and verifies it survives.
- **High: reused request identities.** Install and delete batches shared sequence values. Per-transport reservation separates batches; overflow is rejected. A scripted late-ACK test exercises reuse of one connection.
- **High: unsent ACK acceptance.** The matcher used the whole batch size, so a future sequence could complete an unsent slot and corrupt counters. It now limits matching to sent requests.
- **High: queue deadlock risk.** A blocking producer could stop reading while both socket directions filled. Nonblocking sends and ACK draining handle queue pressure. Scripted EAGAIN and a 5,000-entry window test cover this path.
- **Medium: false or indefinitely deferred timeouts.** No matching ACK was confused with no data. Per-request monotonic expiry handles unrelated traffic and missing responses.
- **Medium: mock identity corruption.** Setting a high-bit occupancy marker inside the key aliased destination addresses. Explicit slot state preserves every key bit.
- **Medium: mock capacity loss.** Deleted slots were never reused. Tombstones are now reusable while probing still checks for duplicates. Churn tests exceed table capacity over time.
- **Medium: input and parser robustness.** Numeric CLI parsing rejects junk, signs, overflow, zero windows, and extra arguments. ACK parsing accepts unaligned input safely and rejects invalid error values; receive truncation is detected.

Remaining limits include lack of policy management, reconciliation, per-UE transactions, rekey/expiry handling, IPv6, secure operational key management, and packet-level interoperability. Deadline scanning can become expensive with many completed entries behind a delayed early request; a production dispatcher should use a bounded pending structure and timer heap or wheel.

Checkpoint: Which finding can damage state owned by another application? Cleanup ownership. Which findings can produce an incorrect result without changing the wrong state? ACK correlation and timeout handling.

10 What the tests establish

The first review validation run passed on Linux: GitHub Actions run 35706266027, source commit 127818d. Both GCC and Clang passed unit/mock tests, AddressSanitizer with UndefinedBehaviorSanitizer, ThreadSanitizer, and CLI checks. A separate job passed actual XFRM install/delete and pre-existing-SA preservation inside an isolated network namespace. See `docs/review.md` for final validation links.

The scripted transport tests reverse ACK order, pack multiple ACKs into a datagram, duplicate an ACK, inject a future sequence, return a stale batch ACK, drop responses, emit unrelated traffic continuously, simulate EAGAIN, and fail a send. These are essential because the normal mock replies in FIFO order and cannot by itself demonstrate out-of-order correctness.

The normal mock tests compare synchronous and asynchronous results, verify duplicate-state errors, delete installed states, distinguish previously aliased keys, and repeatedly install/delete more than 65,536 distinct entries. The large-window test exercises socket backpressure with real Unix sockets and threads.

The real test verifies that the kernel accepts the generated AEAD state message. It then checks the state table is empty after successful benchmark cleanup. Finally it pre-installs the first benchmark identity, expects the benchmark to fail, and confirms that identity still exists. Destroying the namespace contains the experiment even if a test fails.

These tests do not establish encrypted packet exchange, production throughput, restart recovery, IMS conformance, every Linux version, or every architecture. No kernel crash/fault injection campaign has been performed. The first synchronous real operation may trigger crypto initialization, making a tiny smoke-test timing comparison especially misleading.

Exercise: Add a test that reports `-ESRCH` when deleting a missing state. Then design a test for receive truncation. State whether each test uses a fake transport, the threaded mock, or a real kernel and why.

11 Running the laboratory

Use a Linux VM or Linux development environment with a C compiler, make, pthreads, Linux UAPI headers, and iproute2 for the real test. The code is Linux-specific; compiling it as a native Windows program is not supported.

```
make
make test
make asan
make tsan
make clean
make CC=clang test
make xfrm-bench
sh tests/test_cli.sh
./xfrm-bench --count 5000 --window 64 --service-us 5
```

Run sanitizer targets sequentially. Each uses `clean`, and they share the same output name. Parallel invocation such as `make -j asan tsan` can race the builds. Use an external timeout when experimenting with scheduling changes.

For real XFRM, use a disposable network namespace. The following command is also the CI approach; it never needs a global state flush:

```
sudo unshare -n sh tests/test_real.sh
```

For interactive inspection, start a shell in a fresh namespace, install a test state with `iproute2`, inspect it, then leave the shell. Do not use benchmark keys for real traffic. The benchmark normally cleans up its successful installs before returning, so an empty table afterward is expected.

```
sudo unshare -n sh
ip xfrm state list
ip xfrm policy list
cat /proc/net/xfrm_stat
exit
```

EPERM generally points to missing capability or namespace restrictions. Unsupported algorithms or malformed requests need the actual kernel error and configuration checked. A timeout means inspect state and transport behavior; it is not evidence that the requested state is absent. XFRM counters describe packet-processing failures and become useful once packet tests exist [R6].

Checkpoint: Why isolate by network namespace? XFRM state and policy are network-namespace scoped, letting a lab manipulate test state without using the host's operational table.

12 Measuring performance honestly

The benchmark reports successful ACK-confirmed installs divided by elapsed installation time. Cleanup is outside that interval. It compares a window-one batch with a larger window. It does not record individual latency percentiles or SIP registration latency.

For a simple model, let S be serialized service time and R be additional round-trip/coordination overhead. A synchronous stream may cost roughly N times $(S + R)$. Pipelining can overlap waiting, but throughput remains limited by kernel work, syscalls, scheduling, queue capacity, and the application. W is not a multiplier on kernel parallelism.

The threaded mock puts work on another thread. Real XFRM may process the request while `send` is executing, so hiding mock worker delay does not prove the same improvement on the kernel path. This is a key explanation to give in an interview, not a footnote to hide.

```
for w in 1 2 8 32 64 256; do
  ./xfrm-bench --count 20000 --window "$w" --service-us 5
done
```

This CLI always prints the synchronous baseline as well as the chosen-window run. Repeat each configuration, separate warmup from measurement, and retain raw outputs. Record commit, CPU, VM/container restrictions, kernel, compiler, count, window, service time, errors, and whether the transport was real or mock.

The README's original 2.2x/1.8x figures are historical mock observations, not newly reproduced numbers for the corrected implementation. The review changes the mock hash table and scheduling work, so regenerate any performance table rather than comparing different implementations as if nothing changed.

A senior-level next experiment would add per-request send/completion timestamps, p50/p95/p99, CPU utilization, queue-pressure counts, and failures. Then measure complete UE security-context establishment, including policies and rollback. Keep any independently measured employment results separate from repository measurements; this repository does not validate a production 5x claim.

Exercise: Explain how throughput can improve while p99 latency worsens when the window grows.
Answer: more queueing can keep resources busy while increasing the waiting time of individual requests.

13 Extending this toward a service

Treat the current code as a batch library and benchmark. A service needs a persistent event loop and per-operation records. A useful record includes sequence, action, object identity, owner/generation, send time, deadline, and completion callback. A bounded sequence-indexed pending table avoids scanning an ever-growing history of completed work.

Add GETSA/dump reconciliation before automatic retries. Separate desired state from observed kernel state. On restart, rebuild knowledge from kernel state and durable ownership records. Do not replay deletes merely because a previous process timed out. A failed acknowledgement channel is a recovery event, not proof that all objects vanished.

For a UE transaction, track each required state and policy. Define the order that prevents cleartext exposure, the point at which registration may proceed, and compensating actions after a partial failure. Add rekey overlap, expiry notifications, deregistration cleanup, and a generation identifier so late events cannot tear down a replacement context.

Operational key handling requires a real key source, controlled access, lifetime management, and careful logging. Never print raw key material in diagnostics. The deterministic test keys, unlimited limits, and lack of rekey make this repository unsuitable for protecting real traffic. RFC 4106 requires nonce uniqueness for a given key [R4].

For high availability, state replication alone does not establish which node owns an operation. Specify fencing and active ownership, route/namespace placement, restart behavior, and reconciliation. Reusing ESP sequence/key state incorrectly can violate security requirements; HA for IPsec needs more than copying a C structure.

Recommended extension order: policy builders with unit tests; packet tests between namespaces; read/reconcile support; registration transaction/rollback; expiry and rekey; then service metrics and controlled performance studies. Each stage should have its own explicit acceptance criteria.

Design question: Why not immediately add multiple sending threads? They complicate sequence ownership, response dispatch, and transaction ordering. First establish where CPU time is spent and whether the kernel serializes the relevant work.

14 Interview explanation and practice

A factual opening is: "This C project compares synchronous and windowed XFRM SA installation. I use Netlink sequences to correlate results, bound outstanding requests, and test failures with a scripted transport. I also run actual state installation and cleanup in a disposable Linux network namespace. It is a control-plane demonstration; it does not implement packet encryption or the full IMS security lifecycle."

Use first-person statements about changes only after you understand and can defend them. Assistance with implementation does not substitute for knowing why the invariants are correct. Distinguish your original work, this review, and improvements you subsequently build yourself.

- **Why not treat all ACKs in arrival order?** Arrival order is not the operation identity. Match a fresh sequence to a sent pending request.
- **Why can a duplicate be harmful?** Counting it twice can decrement the outstanding count incorrectly and complete the batch early.
- **Why use monotonic time?** Wall-clock corrections should not change timeout duration.
- **Does async mean nonblocking to the caller?** No. These batch functions block until completion/failure. They pipeline internal requests; they do not expose futures or callbacks.
- **What does the kernel do asynchronously?** Do not assume parallel kernel execution. Request processing may occur during send; the application still reads ACKs separately.
- **Why not delete every requested SA afterward?** A rejected request may collide with someone else's object. Cleanup needs ownership evidence.
- **What does TSan prove?** It detects data races on exercised paths; it does not prove deadlock freedom or protocol correctness.
- **What would you implement next?** Policies and packet tests, followed by reconciliation, before claiming production readiness.

Whiteboard drill: Spend five minutes drawing the files and message format; five tracing a reordered ACK batch; five explaining timeout versus rejection; and five proposing reconciliation. Have another engineer challenge the ownership and restart assumptions.

Claim check: This is Linux userspace C calling a kernel API. Use "Linux XFRM control-plane programming" rather than "wrote the Linux IPsec stack" or "implemented a kernel module." Describe real benchmark measurements only with their environment and evidence.

15 A seven session learning plan

Session 1 - Objects and boundaries. Read chapters 1-3 and both headers. Draw state versus policy, distinguish SPI/reqid/Netlink sequence, and describe the project's omissions without notes.

Session 2 - Byte layout. Read the builders and encoding tests. Trace NEWSA and DELSA field assignments. Add assertions for source, destination, reqid, replay window, attribute length, and exact key bytes. Predict what happens when the output buffer is one byte too small for the attribute.

Session 3 - Scheduling. Trace the manager with $W=1$ and $W=3$. Write down the invariants. Use a debugger to observe next, done, inflight, and results while stepping through a short mock batch.

Session 4 - Failure injection. Read the scripted transport. Explain every mode before executing it. Add a reordered mix of success and kernel rejection, then a delayed reply arriving after timeout. Check that the batch cannot finish twice for one request.

Session 5 - Kernel laboratory. Run the real namespace test. Explain why EEXIST preservation matters. Inspect iproute2 state/policy output in a disposable namespace and compare the fields with the UAPI structures. Do not infer encrypted traffic from installation alone.

Session 6 - Measurement. Sweep windows in the mock, keep raw outputs, and repeat runs. Plot successful SA/s versus W and describe the saturation point. Identify what additional instrumentation is needed for p99 latency and registration-level timing.

Session 7 - Design review. Propose a GETSA reconciliation API and a four-state registration transaction. Include timeout, restart, conflicting ownership, and rollback. Explain the proposal aloud, then identify which claims are demonstrated and which remain plans.

You are ready to discuss this project when you can diagnose an ACK-loss timeline, explain the cleanup bug, reproduce the tests, and propose a bounded recovery mechanism without memorizing a script. Senior experience is most visible in the assumptions and failure cases you notice.

16 References and further reading

These references support the protocol and kernel concepts. The repository source and regression tests remain the authority for what this implementation actually does. Versioned Linux source links below are stable reading references, not a claim that CI ran that kernel version.

- [R1] RFC 4301, Security Architecture for the Internet Protocol. Read the SA, security policy database, and security association database sections. <https://www.rfc-editor.org/rfc/rfc4301.html>
- [R2] ETSI TS 133 203 V18.0.0, corresponding to 3GPP TS 33.203 Release 18. Start with clauses 7.1 and 7.2; select the correct access scenario and release for a real implementation. https://www.etsi.org/deliver/etsi_ts/133200_133299/133203/18.00.00_60/ts_133203v180000p.pdf
- [R3] Linux kernel documentation, Introduction to Netlink. Focus on Classic versus Generic, sequences, acknowledgements, and extended ACKs. <https://www.kernel.org/doc/html/latest/userspace-api/netlink/intro.html>
- [R4] RFC 4106, AES-GCM in ESP. Read key/salt, IV/nonce, and authentication-tag requirements. <https://www.rfc-editor.org/rfc/rfc4106.html>
- [R5] Linux v6.12 XFRM UAPI definitions. Locate `xfrm_usersa_info`, `xfrm_usersa_id`, `xfrm_algo_aead`, and `XFRMA_ALG_AEAD`. <https://github.com/torvalds/linux/blob/v6.12/include/uapi/linux/xfrm.h>
- [R6] Linux XFRM transformation statistics. Use this when adding packet-level tests. https://cdn.kernel.org/doc/html/latest/networking/xfrm/xfrm_proc.html
- [R7] Linux v6.12 XFRM userspace request handling. Follow NEWSA handling from the dispatch table through validation and state creation. https://github.com/torvalds/linux/blob/v6.12/net/xfrm/xfrm_user.c
- [R8] Linux v6.12 Netlink UAPI. Read `nlmsg_hdr`, `nlmsgerr`, flags, and alignment macros. <https://github.com/torvalds/linux/blob/v6.12/include/uapi/linux/netlink.h>

Useful source-reading questions: Where is the request checked? Which lock protects insertion? Where is the ACK produced? What differs between a state lookup and a policy lookup? Which data is per network namespace? Answer against a pinned kernel revision, because implementation details evolve.

For the full review and validation record, read `docs/review.md`. For reproduction commands and scope, read the repository README.